

HW1_Report



Author: 111511198 鄭恆安

Concept and Code Details of Custom Preprocessor Class

Overview:

The `Preprocessor` class in `preprocessor.py` is designed to handle essential data preprocessing tasks, specifically for converting and preparing data for machine learning models. It standardizes numerical features and converts categorical features into numerical values, such as transforming 'Yes' and 'No' values into 1 and 0, respectively. This ensures the data is in a format suitable for use in logistic regression classifier. Additionally, the class includes feature selection based on correlation thresholds to help identify and retain relevant features.

Class Structure:

1. Attributes:

- `df (pandas.DataFrame)`: The input will be the DataFrame that holds the data to be preprocessed.

2. Methods:

- `__init__(self, df)`:
 - This method initializes the `Preprocessor` object by accepting a DataFrame `df` as input. The DataFrame is stored as an instance variable for further manipulation.

```
def __init__(self, df):  
    self.df = df
```

- `standardize(self)`:
 - Standardizes numeric features by scaling them to have a mean of 0 and a standard deviation of 1. This is specifically applied to features F1 to F17.
 - Formula for standardization:
$$x_{\text{new}} = \frac{x - \mu}{\sigma}$$

where μ is the mean, and σ is the standard deviation of the feature column.
 - The standardization scales all data between -1 and 1.

```
def standardize(self):
    self.df.iloc[:, 1:18] = (self.df.iloc[:, 1:18] - self.df.iloc[:, 1:18].mean()) / self.df.iloc[:, 1:18].std()
    return self.df
```

- `booleanize(self)`:

- Converts categorical data (features F18 to F77) into binary numeric form. Specifically, the function transforms 'Yes' to 1 and 'No' to 0 using a lambda function.
- This transformation is necessary because machine learning algorithms typically require numeric input, and booleanizing categorical data simplifies the dataset.

```
def booleanize(self):
    for i in range(18, 78):
        self.df.iloc[:, i] = self.df.iloc[:, i].apply(lambda x: 1 if x == 'Yes' else 0)
    return self.df
```

- `drop_column(self, column)`:

- Drops an unwanted column from the DataFrame. This is useful when some columns are deemed unnecessary or irrelevant for model training.

```
def drop_column(self, column):
    self.df = self.df.drop(columns=[column])
    return self.df
```

- `feature_selection(self, threshold)`:

- Selects features based on their correlation with the target label. It computes the absolute correlation between features and the label and retains those whose correlation value is greater than or equal to the given threshold.
- This helps reduce the feature set to only the most relevant features, reducing overfitting and improving model interpretability.

```
def feature_selection(self, threshold):
    column = self.df.corr()['label'][abs(self.df.corr()['label'])
    >= threshold].index
    column = column.tolist()
    return column
```

- `preprocess(self)`:

- Calls the `standardize` and `booleanize` methods to preprocess the DataFrame. This is the main entry point for preparing the data, combining both standardization and

booleanization steps into one function.

```
def preprocess(self):  
    self.standardize()  
    self.booleanize()  
    return self.df
```

Concept and Code detail of dataPreprocessing function:

Code

```
def dataPreprocessing():  
  
    train_X = pd.read_csv('proj1_data/train_X.csv')  
    train_y = pd.read_csv('proj1_data/train_y.csv')  
    test_X = pd.read_csv('proj1_data/test_X.csv')  
    test_y = pd.read_csv('proj1_data/test_y.csv')  
  
    # preprocess the data  
    # standardize & booleanize the data  
    train_X = Preprocessor(train_X).preprocess()  
    test_X = Preprocessor(test_X).preprocess()  
  
    # use feature_selection to select the feature  
  
    # concat train_X and train_y  
    train_data = pd.concat([train_X, train_y], axis=1)  
    train_data = train_data.fillna(0)  
    selected_column = Preprocessor(train_data).feature_selection(0.11)  
    train_data = train_data[selected_column]  
  
    # concat test_X and test_y  
    test_data = pd.concat([test_X, test_y], axis=1)  
    test_data = test_data.fillna(0)  
    test_data = test_data[selected_column]  
  
    # convert the data to numpy array  
    train_X = train_data.iloc[:, :-1].to_numpy().astype(float)  
    train_y = train_data.iloc[:, -1].to_numpy().astype(float)  
  
    test_X = test_data.iloc[:, :-1].to_numpy().astype(float)  
    test_y = test_data.iloc[:, -1].to_numpy().astype(float)
```

```
return train_X, train_y, test_X, test_y
```

Workflow

1. Data Loading:

- Training and testing datasets are read from CSV files.

2. Preprocessing:

- The `Preprocessor` class is used to preprocess both training and testing datasets. Specifically, it:
 - Standardizes numeric features (F1-F17).
 - Converts categorical features (F18-F77) from 'Yes'/'No' to 1/0.

```
train_X = Preprocessor(train_X).preprocess()  
test_X = Preprocessor(test_X).preprocess()
```

3. Feature Selection:

- After preprocessing, feature selection is applied based on a specified correlation threshold (0.11 in this case). The selected features are then used to create the final dataset.

```
selected_column = Preprocessor(train_data).feature_selection(0.11)  
train_data = train_data[selected_column]
```

4. Handling Missing Data:

- Any missing values in the concatenated data (after combining features and labels) are filled with `0`. Since I have already standardized the data, so '0' here equals the mean value of each column for original data.

```
train_data = train_data.fillna(0)  
test_data = test_data.fillna(0)
```

5. Conversion to Numpy Arrays:

- The final processed data is converted to numpy arrays, which are the required input format for most machine learning algorithms.

```
train_X = train_data.iloc[:, :-1].to_numpy().astype(float)  
train_y = train_data.iloc[:, -1].to_numpy().astype(float)  
test_X = test_data.iloc[:, :-1].to_numpy().astype(float)  
test_y = test_data.iloc[:, -1].to_numpy().astype(float)
```

Result of Performance

Acc: 0.77907

Discussion

Initially, no matter how I preprocessed the dataset, my model's predictions were **always all zeros**. After some trial and error, and researching online, I realized that the issue might be due to having too many features but not enough training data for the classifier to learn effectively. To address this, I decided to use feature correlation to select the most important features. Through further experimentation, I found that setting the correlation **threshold to 0.11** resulted in the best accuracy on the test dataset.

All in all, this homework helped me realize the crucial role of data preprocessing. No matter how powerful your machine learning algorithm is, without proper data preprocessing, the training results might not meet your expectations. Preprocessing ensures the data is clean and well-structured, allowing the model to learn more effectively and produce better outcomes.

Threshold Experiment

Threshold	0.05	0.075	0.1	0.11	0.15
Accuracy	0.65116	0.67442	0.75581	0.77907	0.72093

Conclusion:

This custom preprocessor automates the data preparation pipeline, efficiently handling both numeric and categorical data, filling missing values, and selecting relevant features based on correlation. The combination of standardization and booleanization ensures that the dataset is ready for training machine learning models, making the `Preprocessor` class a valuable tool in the data preprocessing stage.